



資料結構

Data Structure

Lab 08

姓名： 黃聖傑

學號： 114310140

Lab08-Q1.

Modify the inorder traversal function to a postorder traversal function in Part1.

Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 樹的節點
class TreeNode {
public:
    int value;           // 節點的值
    TreeNode* left;      // 左子節點
    TreeNode* right;     // 右子節點
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
};

// 樹結構
class BinaryTree {
public:
    TreeNode* root;
    BinaryTree() : root(nullptr) {} // 初始化樹

    // 用陣列構建二元樹
    TreeNode* buildTree(vector<int>& arr) {
        if (arr.empty()) return nullptr;

        queue<TreeNode*> q; // 儲存待處理的節點
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(root); // 將根節點加入 queue

        size_t i = 1; // 陣列索引
        while (!q.empty()) {
            TreeNode* current = q.front(); // 取出隊列中的節點
            q.pop();

            // 添加左子節點
```

```

        if (i < arr.size()) {
            current->left = new TreeNode(arr[i]);
            q.push(current->left); // 將左子節點加入 queue
            i++;
        }
        // 添加右子節點
        if (i < arr.size()) {
            current->right = new TreeNode(arr[i]);
            q.push(current->right); // 將右子節點加入 queue
            i++;
        }
    }

    return root;
}

// 中序遍歷
void inorderTraversal(TreeNode* node) {
    if (node == nullptr) return; // 如果節點為空，忽略

    inorderTraversal(node->left); // 遍歷左子樹
    cout << node->value << " "; // 訪問當前節點
    inorderTraversal(node->right); // 遍歷右子樹
}

// 後序遍歷
void PostorderTraversal(TreeNode* node) {
    if (node == nullptr) return; // 如果節點為空，忽略

    PostorderTraversal(node->left); // 遍歷左子樹
    PostorderTraversal(node->right); // 遍歷右子樹
    cout << node->value << " "; // 訪問當前節點
}

};

int main() {
    BinaryTree tree; // 宣告二元樹

    // 輸入陣列用於構建樹，NULL 表示空子節點

```

```

vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };

tree.buildTree(arr); // 建立樹

// 中序遍歷輸出
cout << "Inorder Traversal: ";
tree.inorderTraversal(tree.root);
cout << endl;
// 後序遍歷輸出
cout << "Postorder Traversal: ";
tree.PostorderTraversal(tree.root);
cout << endl;

return 0;
}

```

Result

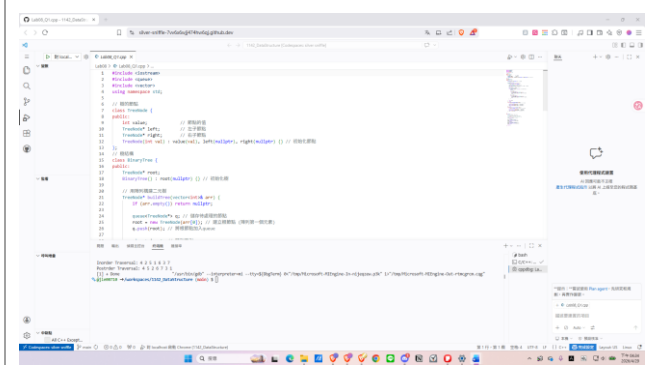
(a)

```

Inorder Traversal: 4 2 5 1 6 3 7
Postorder Traversal: 4 5 2 6 7 3 1

```

(b)



Lab08-Q2.

Implement a method to find the maximum value in the left and right subtrees.

Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 樹的節點
class TreeNode {
public:
    int value;           // 節點的值
    TreeNode* left;      // 左子節點
    TreeNode* right;     // 右子節點
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
};

// 樹結構
class BinaryTree {
public:
    TreeNode* root;
    BinaryTree() : root(nullptr) {} // 初始化樹

    // 用陣列構建二元樹
    TreeNode* buildTree(vector<int>& arr) {
        if (arr.empty()) return nullptr;

        queue<TreeNode*> q; // 儲存待處理的節點
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(root); // 將根節點加入 queue

        size_t i = 1; // 陣列索引
        while (!q.empty()) {
            TreeNode* current = q.front(); // 取出隊列中的節點
            q.pop();

            // 添加左子節點
```

```

        if (i < arr.size()) {
            current->left = new TreeNode(arr[i]);
            q.push(current->left); // 將左子節點加入 queue
            i++;
        }
        // 添加右子節點
        if (i < arr.size()) {
            current->right = new TreeNode(arr[i]);
            q.push(current->right); // 將右子節點加入 queue
            i++;
        }
    }

    return root;
}

// 中序遍歷
void inorderTraversal(TreeNode* node) {
    if (node == nullptr) return; // 如果節點為空，忽略

    inorderTraversal(node->left); // 遍歷左子樹
    cout << node->value << " "; // 訪問當前節點
    inorderTraversal(node->right); // 遍歷右子樹
}

// 尋找 root 的左子樹最大值
int findMaxLeftSubtree(TreeNode* node) {
    if (node == nullptr || node->left == nullptr) {
        return -1; // 如果沒有左子樹，返回-1
    }
    return findMax(node->left); // 在左子樹中尋找最大值
}

// 尋找 root 的右子樹最大值
int findMaxRightSubtree(TreeNode* node) {
    if (node == nullptr || node->right == nullptr) {
        return -1; // 如果沒有右子樹，返回-1
    }
    return findMax(node->right); // 在右子樹中尋找最大值
}

// 在子樹中尋找最大值的輔助函數

```

```

int findMax(TreeNode* node) {
    if (node == nullptr) {
        return -1; // 如果節點為空，返回-1
    }
    int leftMax = findMax(node->left); // 左子樹最大值
    int rightMax = findMax(node->right); // 右子樹最大值
    return max(node->value, max(leftMax, rightMax)); // 返回當前節點值與左右子
    樹最大值中的最大值
}

};

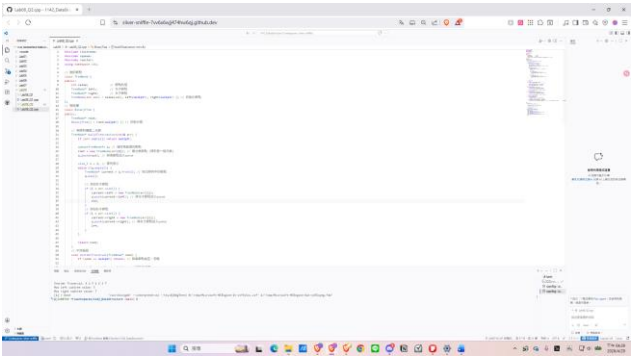
int main() {
    BinaryTree tree; // 宣告二元樹

    // 輸入陣列用於構建樹，NULL 表示空子節點
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };

    tree.buildTree(arr); // 建立樹

    // 中序遍歷輸出
    cout << "Inorder Traversal: ";
    tree.inorderTraversal(tree.root);
    cout << endl;
    //輸出左子樹最大值
    cout << "Max left subtree value: ";
    cout << tree.findMaxLeftSubtree(tree.root) << endl;
    //輸出右子樹最大值
    cout << "Max right subtree value: ";
    cout << tree.findMaxRightSubtree(tree.root) << endl;
    return 0;
}

```

result	
(a)	(b)
<pre>Inorder Traversal: 4 2 5 1 6 3 7 Max left subtree value: 5 Max right subtree value: 7</pre>	

Lab08-Q3.
Explain how to find the maximum value in the left and right subtrees.
Discussion
先以 root 分開左右兩邊的子樹，再透過輔助程式遞迴遍歷左右子樹，每次回傳當前值與其左右子樹這三個值中的最大值，由最下層開始比較，再將最大值傳到上層比較，最後比較出左右子樹中的最大值後不與最高層比較，直接通過 findMaxLeft(Right)Subtree 回傳。

Lab08-Q4.
What happens if the left and right subtrees are not complete trees?
Discussion
比較時有可能比較到空值，遇到空值我設計回傳錯誤訊息-1 表示最大值不存在或子樹不存在，此設定為假設樹裡面的值皆為正，若設定 NULL 的預設值比樹的實際最大值還要大(例如樹裡面可允許存入負數，且有 NULL 存在)，回傳的最大值會是錯誤訊息-1(代表子樹不存在)，而非真正的最大值，如何設計錯誤訊息取決於可存入樹的值的範圍。